Andalusia Academy

Project Standards, Branching Strategy & Code Review

Phase 1 — MVP Graduation Project

1. Purpose

This document defines the minimum standards that all teams must follow while developing the Andalusia Academy project. The goal is to build the system using a clean, consistent, responsive, secure, and professional development approach.

2. Project & Code Standards

2.1 General Code Quality

- Use meaningful names for variables, functions, classes, components, and files.
- Keep functions and components focused on one responsibility.
- Avoid duplicated code; create reusable functions/components when appropriate.
- Do not leave unused imports, variables, commented-out code, or unnecessary files.
- Keep the code readable and consistently formatted.
- Do not hardcode data that should come from the database/API.

2.2 React / TypeScript Standards

- Use React with TypeScript.
- Build reusable components instead of putting large amounts of JSX in one component.
- Separate pages, reusable components, API/service logic, hooks, types, and utilities.
- Use appropriate hooks and avoid unnecessary useEffect calls.
- Handle loading, error, and empty states for API-driven pages.
- Validate forms and display clear validation messages.
- Avoid using any unless there is a justified reason.

2.3 ASP.NET Core / Backend Standards

- Keep controllers focused on HTTP/request-response responsibilities.
- Put business rules and application logic in appropriate services rather than large controllers.
- Use DTOs for API requests/responses where appropriate instead of exposing EF Core entities directly.
- Use validation for incoming data.
- Return appropriate HTTP status codes.
- Protect endpoints with authentication and authorization where required.
- Keep API naming and response formats consistent across the project.

2.4 Database Standards

- Use clear entity and property names.
- Define correct primary keys, foreign keys, relationships, and constraints.
- Do not duplicate data unnecessarily.
- Use EF Core migrations for schema changes.
- Seed suitable development/demo data so the application can be demonstrated consistently.

2.5 Responsive Design — Mandatory

The application must be usable on desktop, tablet, and mobile screens. Students must not design only for their own laptop resolution.

- Navigation must work on mobile.
- Course/program cards must adapt to smaller screens.
- Forms and buttons must remain usable without horizontal overflow.
- Images must resize appropriately.
- Tables and dashboards must have an appropriate mobile presentation.
- No important content should be cut off or overlap at common mobile widths.

2.6 UI/UX Standards

- Keep typography, spacing, buttons, colors, and components visually consistent.
- Use clear labels and feedback for forms.
- Show loading indicators when waiting for API operations.
- Show meaningful error messages when an operation fails.
- Show an appropriate empty state when no data exists.
- Maintain a consistent navigation and user experience throughout the application.

2.7 Security Standards

- Never commit passwords, API keys, JWT secrets, payment secrets, or database credentials to Git.
- Authentication alone is not authorization; protected backend endpoints must enforce permissions.
- Do not return sensitive information such as password hashes in API responses.
- A hidden frontend button is not a security mechanism. Authorization must be enforced on the backend.

3. Git & Branch Organization

The project will use a feature-based branching approach. Each meaningful feature should be developed in its own branch. The main branch should remain stable.
Recommended structure:

Main

|__ Development

├— dev/authentication
├— dev/course-catalog
├— dev/course-details
├— dev /programs
├— dev /career-paths
├— dev /enrollment
├— dev /payment
├— dev /learner-dashboard
├— dev /attendance
├— dev /assignments
├— dev /instructor-dashboard
├— dev /admin-dashboard
└— dev /cms

3.1 Branch Naming

Use: dev /<feature-

name> Examples:

- dev /authentication
- dev /course-catalog
- dev /enrollment
- dev /learner-dashboard For bug fixes, use: fix/<problem-name>

3.2 One Feature = One Branch

Do not create one huge branch containing unrelated features. A branch should have a clear purpose so that it can be reviewed and tested independently.

Bad example: feature/everything

Good examples: feature/authentication, feature/course-catalog, feature/payment

3.3 Feature Development Should Be End-to-End

Frontend and backend should not be developed independently for a long period. When possible, a feature should connect the complete flow:

React → ASP.NET Core API → Business Logic → EF Core → SQL Server

4. Commit Standards

Commit messages should clearly describe the change.

- feat: add course listing endpoint
- feat: implement course search
- fix: correct course pagination
- fix: prevent duplicate enrollment
- refactor: move enrollment logic to service
- test: add enrollment tests
- docs: update API documentation

Avoid vague commits such as: update, changes, test, final, final2, or aaa.

5. Pull Request & Code Review Process

Every completed feature must go through a Pull Request before becoming part of main.

Required workflow:

1. Create a feature branch.
2. Implement the feature.
3. Test the feature locally.
4. Push the branch to GitHub.
5. Create a Pull Request targeting main.
6. Add the instructor as a reviewer.
7. Wait for the review.

8. Fix any requested changes in the same Pull Request.
9. Get instructor approval.
10. Only then merge the Pull Request into main.

5.1 Instructor Reviewer

GitHub: https://github.com/AlyZakaria

Email: ali.zakariya1929@outlook.com

5.2 How to Add the Instructor as Reviewer

Needs first to add the instructor as Collaborators

11. Open your Pull Request on GitHub.
12. Find the Reviewers section in the Pull Request sidebar.
13. Click the option to select/request reviewers.
14. Search for AlyZakaria.
15. Request the review.

5.3 No Merge Without Approval

Important rule: Do not merge the Pull Request into main until the instructor has reviewed and approved it.

If changes are requested, make the corrections, push the new commits to the same branch, and update the same Pull Request. The instructor will review the changes again.

Feature Branch → Dev branch → Pull Request → Instructor Review → Approval → Merge to main

6. Pull Request Requirements

- Use a clear Pull Request title, for example: feat: implement course enrollment.
- Describe what was implemented.
- Mention important frontend, backend, and database changes.
- Mention any known limitations or issues.
- Confirm that the feature was tested locally.
- Resolve review comments before merging.

7. Definition of Done

A feature is considered complete only when the relevant items below have been satisfied:

- Frontend implemented.
- Backend/API implemented.
- Database changes implemented where required.
- Validation implemented.
- Error handling implemented.
- Authentication/authorization implemented where required.
- Responsive on desktop, tablet, and mobile.
- Loading and empty states handled where applicable.
- Tested locally.
- No unnecessary console errors or warnings.

- No sensitive information committed.
- Pull Request created.
- Instructor review completed.
- Requested changes resolved.
- Instructor approval received.
- Pull Request merged into main.

8. Final Expectations

Every team member is responsible for maintaining the quality of the shared codebase. The project should be understandable and maintainable by another developer, not only by the person who originally wrote the code.

Build clean. Build responsive. Build securely. Review before you merge.